



ENGINEERING DOCUMENTATION

DOCUMENT 05 / 09

API Documentation

AI Hiring Platform · Intelligence-First

Generated **June 14, 2026** · Confidential — internal use

Lexy — API Documentation

Engineering documentation set · Doc 5 of 9 The HTTP surface of `@workspace/api-server`. All routes are mounted under `/api`. Validation is Zod (`@workspace/api-zod`); persistence is Drizzle (`@workspace/db`).

1. Authentication

Mechanism	Where used	How it works
Bearer token (primary)	All recruiter/admin/candidate API calls	HMAC-SHA256-signed token of the form <code>v2.<payload>.<signature></code> , resolved by the <code>resolveUser</code> middleware. The frontend stores it in <code>localStorage</code> and <code>apiFetch</code> attaches it.
Session cookie	Candidate live-interview flow	<code>lexy_interview_session</code> , validated by <code>requireInterviewSessionCookie</code> ; binds the session to a browser fingerprint + HMAC. Path-scoped and cleared by <code>/end</code> .
URL tokens	Invites & one-click replies	<code>/api/invites/:token</code> , <code>/api/staff-invites/:token</code> , <code>/api/outreach/reply/:token</code> .

Critical rule — header presence is not authentication. A handler must resolve the caller (`getAuthUserId` + lookup) and **401 a null user before any role check**. Don't trust a header just because it's present.

2. Authorization & tenant scoping

Every authenticated `/api/*` request passes through `withTenantContext` before the handler:

1. Acquire a DB connection, `SET ROLE lexy_app`.
2. Set GUCs `app.current_tenant_id` and `app.allowed_tenant_ids`.
3. Postgres **RLS** policies then restrict visibility on `candidates`, `applications`, `interview_sessions`, etc.

`getAllowedTenantIds(user)` resolves the caller's scope:

Role	Scope
<code>platform_admin</code>	<code>null</code> → sees everything (cross-tenant).
<code>tenant_admin</code>	own tenant + all descendants (recursive CTE subtree).
<code>recruiter</code> , <code>hiring_manager</code> , <code>interviewer</code>	tenant-scoped (per role).
<code>candidate</code>	candidate portal only.

`STAFF_ROLES` = `platform_admin`, `tenant_admin`, `recruiter`, `hiring_manager`, `interviewer`.
`getAllowedTenantIds` **is not a staff gate** — candidates also have a non-null `tenantId`, so staff-only routes need an explicit `STAFF_ROLES` allowlist. Any caller-supplied `jobId` / `candidateId` must be validated against the allowed set **before** use.

■ 3. Conventions

- **Base path:** all endpoints below are relative to `/api`.
- **Validation:** request bodies are whitelisted by Zod — **unknown body keys are silently stripped**. A field your handler reads must be in the route's schema.
- **Errors:** prefer explicit status codes; `409 email_match` for candidate dedup collisions, `401` for unauthenticated, `403` for out-of-scope tenant access.
- **Async work:** long-running operations enqueue an `ai_jobs` row and return quickly; the UI infers “running” from stage + null result + recent `updatedAt`.
- **Rate limiting:** there is **no global, API-wide rate limit**. Limiting is applied **selectively** via `middlewares/rateLimit.ts` to abuse-prone or unauthenticated endpoints only — e.g. login, candidate registration, password reset/forgot, talent-pool apply, invite-accept, and interview start/step-up verification — keyed per IP / per email / per session (typical windows: a handful of requests per minute or per hour). The main authenticated CRUD routes are **not** rate-limited today. **Do not** assume the platform will protect you from a runaway integration: if you're building an automated client, throttle on your side, and don't poll authenticated endpoints aggressively.

⚠ **GET /candidates quirk (read before you build on it).** This endpoint returns the entire candidate bench in a single payload, **capped at 1000 rows**, and it **ignores** `limit` (and offset-style pagination) entirely. Two failure modes for the unwary: (1) you receive a much larger response than expected and assume pagination is happening when it isn't; (2) on a tenant with >1000 candidates you **silently get a truncated list** with no error and no “next page” — code that treats the result as complete will quietly drop candidates. *Why it's like this:* the recruiter UI loads and filters the bench client-side, so the endpoint was built to hand back the whole working set at once rather than page it. If you need guaranteed-complete results above the cap, use `POST /candidates/nl-search` / a scoped query rather than relying on this list.

■ 4. Route inventory by domain

Representative endpoints per domain. Handlers live in `artifacts/api-server/src/routes/*`; this is the map, the code is the contract.

Auth — `auth.ts`

Method	Path	Purpose
POST	<code>/auth/login</code>	Recruiter/admin login (email + password).
POST	<code>/auth/candidate-login</code>	Candidate portal login.
POST	<code>/auth/register</code>	Candidate self-registration.
GET	<code>/auth/me</code>	Current user profile from the token.

Sourcing — `sourcing.ts`

Method	Path	Purpose
POST	<code>/sourcing/search</code>	Multi-source candidate search (internal → github/pdl/serp). Internal-first.
POST	<code>/sourcing/nl-search</code>	Natural-language candidate sourcing.
GET	<code>/sourcing/candidates</code>	List saved sourced candidates for a tenant.

Providers run behind the adapter registry (`lib/sourcing-providers.ts`) with per-kind timeouts; a failed/disabled provider returns a `skipped` reason rather than failing the whole search.

Candidates — `candidates.ts`

Method	Path	Purpose
GET	<code>/candidates</code>	Candidate pool (tenant/platform scoped). ⚠ Known limitation: returns the <i>whole</i> bench in one response, hard-capped at 1000 rows , and ignores the <code>limit</code> query param. See note below.
POST	<code>/candidates/nl-search</code>	AI search over the candidate pool.
GET	<code>/candidates/:id</code>	Single candidate profile.
POST	<code>/candidates/parse-cv</code>	Stateless CV text extraction.

Jobs — `jobs.ts`

Method	Path	Purpose
GET	<code>/jobs</code>	List jobs (tenant-scoped).
POST	<code>/jobs</code>	Create job / work order.
POST	<code>/jobs/generate-jd</code>	AI-generate a job description.
PATCH	<code>/jobs/:id/platform-recommendations</code>	Toggle AI cross-tenant recommendation consent.

Pipeline — `pipeline.ts`

Method	Path	Purpose
GET	<code>/jobs/:jobId/pipeline</code>	Pipeline config + recent runs.
POST	<code>/jobs/:jobId/pipeline/run</code>	Trigger async pipeline run (ICP→Sourcing→...).
GET	<code>/jobs/:jobId/pipeline-stages</code>	Kanban data (applications + sourced leads).
PATCH	<code>/sourced-candidates/:id</code>	Advance candidate stage (drag-and-drop).

Interviews — `interviews.ts`

Method	Path	Purpose
POST	<code>/interviews/plans</code>	Create an AI interview plan for a job.
POST	<code>/interviews/sessions</code>	Create a session + issue a candidate invite.
POST	<code>/interviews/transcribe</code>	Real-time STT for audio chunks (Azure/Whisper).
POST	<code>/interviews/:id/converse</code>	Real-time AI interview turn (LLM + TTS).

Entering the `verification` stage auto-runs the Verification Agent for **all** candidates (sourced + applied). Interview `/end` AI-grades each answer once and advances the application stage.

Outreach — `outreach.ts`

Method	Path	Purpose
POST	<code>/outreach/campaigns</code>	Create an automated outreach campaign.
POST	<code>/outreach/messages/:id/approve</code>	Approve an AI-drafted first-touch email.
GET	<code>/outreach/inbox</code>	Recruiter inbox of candidate replies.
GET/POST	<code>/outreach/reply/:token</code>	One-click candidate reply (self-decline / interest).

Default is **approval-required**: drafts wait for a recruiter unless autopilot is explicitly enabled. Draft status is guarded inside the UPDATE predicate to prevent double-sends.

Intelligence — `intelligence.ts`

Method	Path	Purpose
GET	<code>/intelligence</code>	Candidate fit scores + hire probabilities. Requires the bearer token — a missing token 401s and silently dumps every candidate into “Unmatched.”
GET	<code>/intelligence/policy</code>	Manage tenant AI decision thresholds.
POST	<code>/intelligence/:jobId/:candidateId/decide</code>	AI next-best-action recommendation.

Admin / Platform

File	Method	Path	Purpose
<code>tenants.ts</code>	POST	<code>/tenants</code>	Provision a new tenant (<code>platform_admin</code>).
<code>tenants.ts</code>	PATCH	<code>/tenants/:id/billing</code>	Manual billing override (status / paidThroughAt).
<code>system-errors.ts</code>	GET	<code>/system-errors</code>	Diagnostic dashboard for crashes/500s.
<code>audit.ts</code>	GET	<code>/audit</code>	SOC2 audit trail of recruiter/AI actions.

Subscriptions & Billing

File	Method	Path	Purpose
<code>plans.ts</code>	GET	<code>/plans/catalog</code>	Public plan pricing + limits.
<code>plans.ts</code>	POST	<code>/plans/start-trial</code>	Email-verified self-serve demo signup.
<code>billing.ts</code>	POST	<code>/billing/checkout</code>	Stripe Checkout session for upgrade.

Connection Engine (feature-flagged additive module)

Enabled via `ENABLE_CONNECTION_ENGINE` / `ENABLE_CANDIDATE_CONNECTION_ENGINE`.

Method	Path	Purpose
GET	/connection-score/:candidateId	Employer-side engagement score (0-100).
GET	/connection-events/:candidateId	Underlying engagement events.
POST	/connection-event	Record an engagement event.
POST	/connection-score/recalculate	Recompute a connection score.

■ 5. Discovering the rest

This is the high-traffic surface, not an exhaustive OpenAPI dump. The authoritative spec is `@workspace/api-spec` (Orval), from which `@workspace/api-client-react` hooks are generated. To find any endpoint:

```
rg "router\.(get|post|patch|put|delete)\(\" artifacts/api-server/src/routes
```

When you add or change an endpoint: update the Zod schema (`@workspace/api-zod`), regenerate the client (`@workspace/api-spec` codegen), and keep the auth/tenant gating rules in §2 — they are not optional.